MongoDB with Express — Reference Guide

Using the native MongoDB driver in an Express app — connection, collections, CRUD, find(), and REST routes (ES module imports, .then()/.catch() promise handling).

Examples use import syntax, so add "type": "module" to package.json. They assume a db variable (created in the first example) and a books collection.

1. Database Operations

Create / Connect to Database

The native driver connects with a MongoClient. Calling client.db(name) selects a database — there is no separate create command, so the database is only created on disk once a document is written to one of its collections.

Syntax

```
const client = new MongoClient(<connectionString>);
client.connect().then(...).catch(...);
const db = client.db(<databaseName>);
```

Example

```
import { MongoClient, ObjectId } from "mongodb";

const client = new MongoClient("mongodb://localhost:27017");

client
  .connect()
  .then(() => {
     console.log("Connected to MongoDB");
     const db = client.db("libraryDB");
     // libraryDB is only created once a document is written to it
  })
  .catch((err) => console.error(err));
```

Delete Database

db.dropDatabase() permanently deletes the selected database, including all of its collections and documents. This action cannot be undone.

Syntax

```
db.dropDatabase()
```

Example

```
db.dropDatabase()
  .then(() => console.log("libraryDB dropped"))
  .catch((err) => console.error(err));
```

2. Collection Operations

Create Collection

db.createCollection() explicitly creates a collection. It is optional for most apps, since MongoDB creates a collection automatically on the first insert — but it lets you set options such as validation rules or a capped size.

Syntax

```
db.createCollection(<name>, <options>)
```

Example

```
db.createCollection("books")
  .then(() => console.log("books collection created"))
  .catch((err) => console.error(err));
```

Delete Collection

collection.drop() permanently removes a collection along with its documents and indexes. This action cannot be undone.

Syntax

```
db.collection(<name>).drop()
```

Example

```
db.collection("books")
  .drop()
  .then(() => console.log("books collection dropped"))
  .catch((err) => console.error(err));
```

3. CRUD Operations on Document

Create — insertOne()

Inserts a single document. The result carries insertedId, the _id MongoDB generated for the new document.

Syntax

```
db.collection(<name>).insertOne(<document>)
```

Example

```
db.collection("books")
  .insertOne({
    title: "Clean Code",
    author: "Robert Martin",
    price: 450
  })
```

```
  .then((result) => console.log(result.insertedId))
  .catch((err) => console.error(err));
```

Create — insertMany()

Inserts several documents in one call, passed as an array. The result carries insertedCount and an insertedIds map.

Syntax

```
db.collection(<name>).insertMany([<doc1>, <doc2>, ...])
```

Example

```
db.collection("books")
  .insertMany([
    { title: "Refactoring", author: "Martin Fowler", price: 520 },
    { title: "The Pragmatic Programmer", price: 480 }
  ])
  .then((result) => console.log(result.insertedCount))
  .catch((err) => console.error(err));
```

Read — find()

Returns a cursor over every document matching the filter. Unlike Mongoose, the native driver does not resolve to an array on its own — call toArray() to read the results. An empty filter matches every document.

Syntax

```
db.collection(<name>).find(<filter>, <options>).toArray()
```

Example

```
db.collection("books")
  .find({ author: "Robert Martin" })
  .toArray()
  .then((books) => console.log(books))
  .catch((err) => console.error(err));
// Returns every book by that author
```

Read — findOne()

Returns the first document matching the filter, or null if nothing matches.

Syntax

```
db.collection(<name>).findOne(<filter>, <options>)
```

Example

```
db.collection("books")
  .findOne({ title: "Clean Code" })
```

```
  .then((book) => console.log(book))
  .catch((err) => console.error(err));
// Returns the first match, or null
```

Update — updateOne()

Updates the first document matching the filter using update operators such as $set. The result reports matchedCount and modifiedCount.

Syntax

```
db.collection(<name>).updateOne(<filter>, <update>)
```

Example

```
db.collection("books")
  .updateOne(
    { title: "Clean Code" },
    { $set: { price: 499 } }
  )
  .then((result) => console.log(result.modifiedCount))
  .catch((err) => console.error(err));
```

Update — updateMany()

Applies the same update to every document matching the filter.

Syntax

```
db.collection(<name>).updateMany(<filter>, <update>)
```

Example

```
db.collection("books")
  .updateMany(
    { author: "Robert Martin" },
    { $set: { inStock: true } }
  )
  .then((result) => console.log(result.modifiedCount))
  .catch((err) => console.error(err));
```

Delete — deleteOne()

Deletes the first document matching the filter. The result reports deletedCount.

Syntax

```
db.collection(<name>).deleteOne(<filter>)
```

Example

```
db.collection("books")
```

```
  .deleteOne({ title: "Refactoring" })
  .then((result) => console.log(result.deletedCount))
  .catch((err) => console.error(err));
```

Delete — deleteMany()

Deletes every document matching the filter. An empty filter ({}) deletes all documents without dropping the collection.

Syntax

```
db.collection(<name>).deleteMany(<filter>)
```

Example

```
db.collection("books")
  .deleteMany({ price: { $lt: 100 } })
  .then((result) => console.log(result.deletedCount))
  .catch((err) => console.error(err));
// Removes every book priced under 100
```

4. More About find()

find() with a Query Filter

A filter narrows results using field values or query operators such as $gt, $gte, $lt, $in, $or. An empty filter matches every document.

Syntax

```
db.collection(<name>).find({ <field>: <value> })
```

Example

```
db.collection("books")
  .find({ price: { $gte: 500 } })
  .toArray()
  .then((books) => console.log(books))
  .catch((err) => console.error(err));
// Returns books priced 500 or more
```

find() with Projection

The projection option controls which fields come back — 1 to include a field, 0 to exclude it. In the native driver it is passed inside the options object (or chained with .project()).

Syntax

```
db.collection(<name>).find(<filter>, { projection: { <field>: 1 } })
```

Example

```
db.collection("books")
  .find({}, { projection: { title: 1, price: 1, _id: 0 } })
  .toArray()
  .then((books) => console.log(books))
  .catch((err) => console.error(err));
// Returns only title and price for every book
```

find() with sort()

sort() orders the results: 1 for ascending, -1 for descending. It chains onto the cursor before toArray().

Syntax

```
db.collection(<name>).find(<filter>).sort({ <field>: 1 | -1 })
```

Example

```
db.collection("books")
  .find()
  .sort({ price: -1 })
  .toArray()
  .then((books) => console.log(books))
  .catch((err) => console.error(err));
// Most expensive first
```

find() with limit() and skip()

limit() caps the number of documents returned and skip() omits the first n — commonly combined for pagination.

Syntax

```
db.collection(<name>).find(<filter>).skip(<n>).limit(<n>)
```

Example

```
db.collection("books")
  .find()
  .sort({ price: 1 })
  .skip(10)
  .limit(5)
  .toArray()
  .then((page) => console.log(page))
  .catch((err) => console.error(err));
// Skips the first 10 results, returns the next 5
```

Finding by _id with ObjectId

MongoDB stores _id as an ObjectId, not a string, so a string id taken from a URL must be wrapped in new ObjectId() before it can match. Unlike Mongoose, the native driver has no findById() shortcut.

Syntax

```
db.collection(<name>).findOne({ _id: new ObjectId(<idString>) })
```

Example

```
db.collection("books")
  .findOne({ _id: new ObjectId("64f1a2b3c4d5e6f7a8b9c0d1") })
  .then((book) => console.log(book))
  .catch((err) => console.error(err));
// Returns the document with that _id, or null
```

5. Using MongoDB in Express Routes

Setup — Connect, Then Listen

Create one MongoClient for the whole app and start the server only after the connection succeeds. Add express.json() so request bodies are parsed, and reuse the same db variable in every route.

Syntax

```
client.connect().then(() => app.listen(<port>)).catch(...);
```

Example

```
import express from "express";
import { MongoClient, ObjectId } from "mongodb";

const app = express();
app.use(express.json());

const client = new MongoClient("mongodb://localhost:27017");
const db = client.db("libraryDB");

client
  .connect()
  .then(() => {
    Console.log(“Successfully connected to MongoDB”);
  })
  .catch((err) => console.error(err));
//Routes
//…
app.listen(3000, () => console.log("Server on port 3000"));
```

GET — List Books

Reads documents with find().toArray() and returns them as JSON. Query parameters from req.query can drive the filter and pagination.

Syntax

```
app.get(<path>, (req, res) => { ... })
```

Example

```
app.get("/books", (req, res) => {
  const page = Number(req.query.page) || 1;
  const limit = Number(req.query.limit) || 10;
  const filter = req.query.author
    ? { author: req.query.author }
    : {};

  db.collection("books")
    .find(filter)
    .skip((page - 1) * limit)
    .limit(limit)
    .toArray()
    .then((books) => res.json(books))
    .catch((err) => res.status(500).json({ error: err.message }));
});
```

GET — One Book by id

Route parameters arrive as strings, so validate with ObjectId.isValid() and convert with new ObjectId() before querying. Respond 400 for a malformed id and 404 when nothing matches.

Syntax

```
app.get("/books/:id", (req, res) => { ... })
```

Example

```
app.get("/books/:id", (req, res) => {
  const { id } = req.params;
  if (!ObjectId.isValid(id)) {
    return res.status(400).json({ error: "Invalid id" });
  }

  db.collection("books")
    .findOne({ _id: new ObjectId(id) })
    .then((book) => {
      if (!book) {
        return res.status(404).json({ error: "Not found" });
      }
      res.json(book);
    })
    .catch((err) => res.status(500).json({ error: err.message }));
});
```

POST — Create a Book

Reads the new document from req.body, inserts it with insertOne(), and responds 201 Created with the generated _id.

Syntax

```
app.post("/books", (req, res) => { ... })
```

Example

```
app.post("/books", (req, res) => {
  db.collection("books")
    .insertOne(req.body)
    .then((result) => {
      res.status(201).json({ _id: result.insertedId, ...req.body });
    })
    .catch((err) => res.status(500).json({ error: err.message }));
});
```

PUT — Update a Book

Updates the matching document with $set using the fields in req.body. A matchedCount of 0 means no document has that id, so respond 404.

Syntax

```
app.put("/books/:id", (req, res) => { ... })
```

Example

```
app.put("/books/:id", (req, res) => {
  const { id } = req.params;
  if (!ObjectId.isValid(id)) {
    return res.status(400).json({ error: "Invalid id" });
  }

  db.collection("books")
    .updateOne({ _id: new ObjectId(id) }, { $set: req.body })
    .then((result) => {
      if (result.matchedCount === 0) {
        return res.status(404).json({ error: "Not found" });
      }
      res.json({ message: "Updated" });
    })
    .catch((err) => res.status(500).json({ error: err.message }));
});
```

DELETE — Remove a Book

Deletes the matching document. A deletedCount of 0 means nothing matched (404); on success, 204 No Content returns an empty body.

Syntax

```
app.delete("/books/:id", (req, res) => { ... })
```

Example

```
app.delete("/books/:id", (req, res) => {
  const { id } = req.params;
  if (!ObjectId.isValid(id)) {
    return res.status(400).json({ error: "Invalid id" });
```

```
  }

  db.collection("books")
    .deleteOne({ _id: new ObjectId(id) })
    .then((result) => {
      if (result.deletedCount === 0) {
        return res.status(404).json({ error: "Not found" });
      }
      res.status(204).end();
    })
    .catch((err) => res.status(500).json({ error: err.message }));
});
```